FWP SYSTEM: MULTI-AGENT DOUBLE CHECK BUG REPORT

Target: C:\Users\Admin\Desktop\fwp | **Date:** October 2026 | **Execution:** 10-Agent Parallel Swarm

EXECUTIVE SUMMARY: Ten specialized autonomous subagents were spun up to perform an adversarial, double-checking audit across Frontend, Backend (NestJS), ML Pipeline, Database, Concurrency, and Infrastructure. The audit corroborated multiple **High & Critical severity runtime bugs** that break core functionality in production:
• **Frontend:** Hardcoded corrupted Authorization header (Bearer ***}) completely breaking API communications.
• **ML Pipeline:** Data leakage and broken temporal validation split (slicing unsorted data), plus severe train/serving skew on Age and Interests.
• **Backend & Concurrency:** Race conditions on mutual swiping leading to HTTP 500 errors or duplicate/lost matches; non-atomic PostGIS updates and OTP verification.
• **Infra & Testing:** Zero backend test specs causing npm test to fail with exit code 1; unauthenticated Redis/DB exposures in default compose.

1. Audit Agent Coverage & Consensus Summary

Agent / Specialization	Key Focus	Status	Top Corroborated Findings
1. Frontend Audit	Web UI & WS	CONFIRMED	Corrupted Auth Bearer token, missing userId on login, form omissions
2. Backend Audit	NestJS APIs	CONFIRMED	Inactive JWTs bypass, OTP concurrency race, QueryBuilder column mixup
3. ML Pipeline Audit	Models & Skew	CONFIRMED	Temporal split sliced pre-sorted array; Age/Interest training-serving skew
4. Data & DB Audit	TypeORM & SQL	CONFIRMED	Missing initial migration, racy mutual likes, split location transactions
5. Concurrency Audit	Races & Locks	CONFIRMED	Check-then-insert match duplication; Android runBlocking thread stall
6. Tests & CI Audit	Coverage & Build	CONFIRMED	0 unit tests causing npm test failure; missing CI workflow pipeline
7. Infra & Docker	Deployment/Env	CONFIRMED	Exposed DB/Redis ports, static default secrets, un-reloaded ML container
8. Integration Audit	E2E Contracts	CONFIRMED	Faculty UUID vs code mismatch, hot-reload absent in retrain loop
9. Adversarial Audit	Edge Cases/Fuzz	CONFIRMED	Chat limit=NaN 500 error, wildcard CORS + credentials rejection
10. Security Audit	Auth & Policies	FILTERED	Triggered safety provider filter (covered by Adversarial & Backend agents)

2. Deep Dive: High & Critical Verified Bugs

[BUG-01] [CRITICAL] Frontend Corrupted Bearer Token Breaks All Authenticated Calls

Location: frontend/app.js (Line 2)

Evidence & Root Cause: Code defines: const authHeaders=()=>({Authorization:`Bearer ***}`});. The template string hardcodes *** instead of ${token} (likely from accidental regex/scrubbing). Every authenticated HTTP request (profile, chat rooms, photos) immediately returns 401 Unauthorized.

Impact: Web frontend is completely non-functional for logged-in users.

Recommended Fix: Restore variable interpolation: const authHeaders=()=>({Authorization:`Bearer ${token}`}); and ensure tokens remain isolated.

[BUG-02] [CRITICAL] ML Temporal Validation Split Slices Unsorted Arrays (Future Data Leakage)

Location: ml/src/models/train.py (Lines 69-83)

Evidence & Root Cause: Arrays are initialized before sorting: X = df[FEATURE_COLUMNS].values, y = df[label_col].values. Then df = df.sort_values('created_at') sorts the DataFrame, but slicing uses the original pre-sort arrays: X[:train_end]. Temporal order is violated; future events contaminate the training partition.

Impact: Reported offline metrics (NDCG, AUC, MAP) are completely invalid and unrepresentative of production performance.

Recommended Fix: Sort the DataFrame by parsed created_at and reset indices BEFORE extracting X and y.

[BUG-03] [HIGH] Concurrent Mutual Likes Cause Race Condition and Duplicate/Lost Matches

Location: backend/src/swipe/swipe.service.ts (Lines 72-125)

Evidence & Root Cause: Swipe deduplication check findOne is outside the transaction. In addition, when two users like each other simultaneously under PostgreSQL READ COMMITTED, both transactions query for reciprocal likes before the other commits. Both may fail to see the reciprocal row, resulting in swiping recorded but match creation permanently bypassed.

Impact: Users who mutually like each other never get matched, or concurrent retries trigger unhandled 500 UniqueConstraint errors.

Recommended Fix: Catch PostgreSQL error code 23505 and map to ConflictException; insert matches using ON CONFLICT DO NOTHING with row locks.

[BUG-04] [HIGH] Severe Training vs. Serving Feature Skew (Age and Interest Matching)

Location: ml/src/features/build_features.py vs ml/src/serving/features.py

Evidence & Root Cause: 1. **Age Skew:** Training approximates age as academic_year + 17, while serving computes exact age from date_of_birth.
2. **Interest Skew:** Training uses string interest names (Jaccard), while serving compares UUID strings (interest_id), resulting in Jaccard similarity evaluating to 0.0 at inference.

Impact: The trained model weights are applied to corrupted feature distributions in production, causing degraded or arbitrary recommendations.

Recommended Fix: Unify feature extraction helpers into a shared contract; export date_of_birth for training and use matching UUID/name sets across both sides.

[BUG-05] [HIGH] Retrained ML Models are Never Hot-Reloaded into Serving Container

Location: docker-compose.yml (Lines 91-128) & ml/scripts/retrain.py

Evidence & Root Cause: ml-retrainer writes new model weights to shared volume artifacts/active_model, but never notifies ml-service:8000/v1/reload. The serving process loads model.pkl once during startup and continues serving stale models indefinitely.

Impact: Automated retraining pipelines appear successful in DB logs, but users receive predictions from old in-memory models until container restart.

Recommended Fix: In retrain.py, invoke an authenticated POST /v1/reload trigger upon registering a new model version.

[BUG-06] [HIGH] Location Coordinates and PostGIS Point Stored Non-Atomically

Location: backend/src/location/location.service.ts (Lines 14-38)

Evidence & Root Cause: updateLocation updates scalar lat/long via TypeORM save() and then executes a separate raw SQL update for ST_SetSRID(ST_MakePoint(...)). If the second query fails or concurrent updates interleave, coordinates and geography desynchronize.

Impact: Spatial queries and nearby candidate discovery return erroneous distance calculations.

Recommended Fix: Combine coordinate update and PostGIS point generation into a single atomic SQL upsert.

[BUG-07] [HIGH] Backend Has Zero Spec Files; 'npm test' Hard-Fails with Exit Code 1

Location: backend/package.json (Line 13)

Evidence & Root Cause: Jest is configured with testRegex: ".*\.spec\.ts$", but there are 0 .spec.ts files across 58 backend TypeScript files. Executing npm test immediately fails with 'No tests found, exiting with code 1'.

Impact: Prevents standard CI pipelines from passing and leaves critical business flows (Auth, Swipe, Chat) without automated regression shields.

Recommended Fix: Add unit/integration test suites for auth and swipe services; configure --passWithNoTests as a temporary bridge.

[BUG-08] [MEDIUM] Chat Message Query Accepts NaN Limit, Crashing TypeORM QueryBuilder

Location: backend/src/chat/chat.controller.ts (Lines 57-59)

Evidence & Root Cause: @Query('limit') limit?: number has no validation pipe. Passing non-integer query params (e.g. ?limit=abc) coerces to NaN, passed directly to .take(limit), which triggers an unhandled SQL syntax error (500).

Impact: DoS vulnerability on chat history endpoints and unhandled server 500 errors.

Recommended Fix: Enforce ParseIntPipe, @IsInt(), @Min(1), and @Max(100) with safe server-side clamping.

[BUG-09] [MEDIUM] Android Repository Uses Synchronous runBlocking on DataStore

Location: android/app/src/main/java/vn/edu/ptit/fwp/data/FwpRepository.kt (Lines 27-46)

Evidence & Root Cause: OkHttp auth interceptor and Authenticator invoke kotlinx.coroutines.runBlocking { session.accessToken() } on OkHttp dispatcher threads, blocking connection pool workers during I/O.

Impact: Risk of thread starvation, UI freezing, and Android ANR (Application Not Responding) under poor network conditions.

Recommended Fix: Maintain an in-memory token cache updated via asynchronous Flows/Coroutines and serialize token refresh calls with a single Mutex.

[BUG-10] [MEDIUM] Default Docker Compose Exposes Database & Unauthenticated Redis

Location: docker-compose.yml (Lines 11-30, 100-101)

Evidence & Root Cause: Host ports 5432, 6379, and 8000 are directly published. Redis has no password requirement, and Postgres defaults to a hardcoded password (fwp_password_change_me).

Impact: If deployed on a public host or open network without external firewalling, internal databases and caches are exposed to unauthorized access.

Recommended Fix: Isolate databases within the internal Docker network; remove host port mapping from production compose.

3. Recommended Remediation Roadmap

Phase	Target Action Items	Estimated Effort
P0 (Immediate)	• Fix frontend authHeaders token interpolation in app.js. • Fix train.py temporal split indexing before model re-training. • Add 23505 duplicate constraint mapping in swipe.service.ts.	< 2 Hours
P1 (Short-term)	• Synchronize ML training/serving features (age calculation & interest UUID resolution). • Implement /v1/reload hook in retrain.py. • Wrap updateLocation in atomic upsert.	1 Day
P2 (Hardening)	• Add core unit/integration tests in backend (Swipe & Auth) and set up CI pipeline. • Refactor Android token refresh to use Mutex/cache instead of runBlocking. • Secure Docker compose network isolation and remove public DB ports.	2-3 Days